

97. La palette : un indice n'est pas une nuance

Changer la palette change TOUT l'écran d'un coup — sans toucher à une seule tuile.

1. Ce qu'il faut comprendre

Entre la tuile et l'écran, il y a la **palette** : une table de quatre cases qui dit, pour chaque indice 0–3, quelle nuance montrer. `paletteFond(n0, n1, n2, n3)` la remplit : l'indice 0 prendra la nuance `n0`, l'indice 1 la nuance `n1`, et ainsi de suite.

D'origine, c'est `paletteFond(0, 1, 2, 3)`. `paletteFond(3, 2, 1, 0)` **inverse** tout : le clair devient sombre, comme un négatif. `paletteFond(0, 0, 3, 3)` écrase les nuances du milieu : un contraste brutal.

Remarque ce qui change **aussi** : le texte. Les lettres sont des tuiles comme les autres, dessinées en indice 3 sur un fond d'indice 0. Tout ce qui est sur le fond passe par la même palette.

La palette tient dans **un seul octet** de la console — deux bits par indice : `11 10 01 00` en binaire, soit 228, pour la palette d'origine. La changer, c'est écrire un octet : ça ne coûte presque rien, et **ça n'attend rien** : `texte()`, lui, attend une pause de la console avant chaque lettre (chapitre 3). On peut donc la changer à chaque image sans ralentir le jeu.

C'est la clé de tous les effets de la leçon suivante : on ne redessine rien, on change la **lecture** du dessin.

À toi : avec SELECT, fais `paletteFond(0, 0, 0, 0)` — que reste-t-il à l'écran ?

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <paletteFond>` → [60.8. La fonction paletteFond\(\) — les quatre nuances du fond](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **630 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	115 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	85 o	← démarrage — copie les lettres et les dessins en mémoire vidéo

```

EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette      37 o ← bouton() – lit les huit boutons
Tuiles           224 o ← CopierTuiles – 13 lettres sur 43, et 1 tuile dessinée
"A: NEGATIF"     10 o ← texte() – un texte gravé
"B: CONTRASTE"   12 o ← texte() – un texte gravé

```

Lettres gravées : ABCEFGINORST:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte()          x 2    22 o sur place, dans main() + EcrireTexte (10 o)
ecran()          x 2    21 o sur place, dans main() + AttendreVBlank (16 o)
poser()          x 1    13 o sur place, dans main() + AdresseCase (10 o)
paletteFond()    x 3    12 o sur place, dans main()
bouton()         x 2    10 o sur place, dans main() + LireManette (37 o)
image()          x 1     3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : une bande de 4 nuances. Tiens A : elle passe en négatif.
// Tiens B : il ne reste que 2 nuances. Rien : les nuances d'origine.
//
// Ce qui est nouveau : la PALETTE du fond.
// Une tuile ne contient pas des nuances, mais des INDICES (0 à 3).
// La palette dit quelle nuance montrer pour chaque indice :
// paletteFond(n0, n1, n2, n3) → l'indice 0 montre la nuance n0,
//                               l'indice 1 la nuance n1, etc.
// On change la palette : tout l'écran change, sans redessiner une seule tuile.

// La bande : 2 rangées d'indice 0, 2 de 1, 2 de 2, 2 de 3.

#include <Tuile>          // un dessin de 8 × 8 pixels
#include <ecran>          // éteint ou rallume l'écran
#include <texte>          // écrit un texte à l'écran
#include <poser>          // pose une tuile sur une case du fond
#include <bouton>         // lit un bouton de la manette
#include <paletteFond>    // choisit les quatre nuances du fond

```

2. UN DESSIN — BANDES

```

Tuile BANDES = {
    "00000000", "00000000", "11111111", "11111111",
    "22222222", "22222222", "33333333", "33333333",
};

```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Dessin écran éteint.
    texte(1, 1, "A: NEGATIF");
    texte(1, 2, "B: CONTRASTE");
    for (uint8_t c = 0; c < 20; c++) {      // Les 20 colonnes de la ligne 7 :
        poser(c, 7, BANDES);               // la bande de nuances.
    }
    ecran(1);                               // On rallume.

    while (true) {                           // La boucle du jeu :
        image();

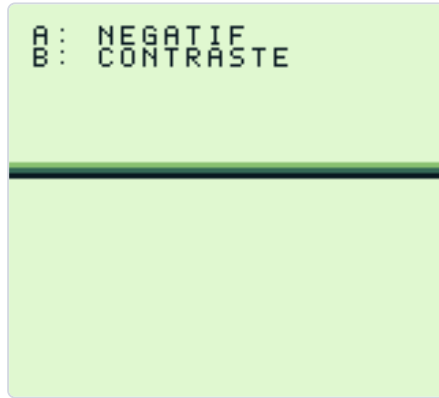
        if (bouton(A)) {                   // A tenue :
            paletteFond(3, 2, 1, 0);        // le négatif : 0 devient 3, 1 devient 2, ...
                                            // (le clair devient sombre ; le texte aussi s'inverse)
        } else if (bouton(B)) {            // sinon, B tenue :
            paletteFond(0, 0, 3, 3);        // deux nuances seulement : 0 et 1 → clair, 2 et 3 → sombre
        } else {                           // sinon, aucune des deux :
            paletteFond(0, 1, 2, 3);        // la palette d'origine : chaque indice montre sa nuance
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

A: NEGATIF
B: CONTRASTE

A: NEGATIF
B: CONTRASTE

A — l'écran n'a pas bougé

B — l'écran n'a pas bougé

5. Ce qu'on doit voir

Quatre bandes ; A les inverse (et le texte avec), B n'en garde que deux nuances.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ au repos : 0, 1, 2, 3 (0123)

✓ A inverse tout : 3, 2, 1, 0 (3210)

✓ le fond du texte s'inverse aussi

✓ B écrase les nuances du milieu : 0, 0, 3, 3 (0033)

✓ au relâché, la palette d'origine revient

Ensuite : **98. Un fondu au noir, et retour** — chapitre 15 : Les quatre nuances : la Game Boy d'origine. · [gameboy3](#) — écrire une cartouche Game Boy en C++